

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# --- Step 1: Create a high-frequency synthetic image ---
# A grid of alternating black and white lines/squares to simulate high frequency
size = 200
high_freq_img = np.zeros((size, size), dtype=np.uint8)
for i in range(0, size, 4):
    high_freq_img[i:i+2, :] = 255 # Horizontal lines
for j in range(0, size, 4):
    high_freq_img[:, j:j+2] = 255 # Vertical lines

# --- Step 2: Simulate aliasing by downsampling without pre-filtering ---
# Downsample the image significantly (e.g., to 1/4th of its original size)
# This acts as undersampling, leading to aliasing
alias_img = cv2.resize(high_freq_img, (size // 4, size // 4), interpolation=cv2.INTER_NEAREST)

# --- Step 3: Corrective approach: Apply a low-pass filter (Gaussian blur) before downsampling ---
# Blurring the image removes high frequencies that would cause aliasing
blurred_img = cv2.GaussianBlur(high_freq_img, (5, 5), 0) # Kernel size (5,5), sigma=0
anti_alias_img = cv2.resize(blurred_img, (size // 4, size // 4), interpolation=cv2.INTER_LINEAR)

# --- Step 4: Downsample the pre-filtered image ---
# Now, downsample the blurred image. Aliasing should be significantly reduced.
anti_alias_img = cv2.resize(blurred_img, (size // 4, size // 4), interpolation=cv2.INTER_LINEAR)

# --- Step 5: Display the results ---
plt.figure(figsize=(15, 5))

plt.subplot(131), plt.imshow(high_freq_img, cmap='gray')
plt.title('Original High-Frequency Image')
plt.axis('off')

plt.subplot(132), plt.imshow(alias_img, cmap='gray')
plt.title('Aliased Image (Undersampled)')
plt.axis('off')

plt.subplot(133), plt.imshow(anti_alias_img, cmap='gray')
plt.title('Anti-Aliased Image (Blurred then Undersampled)')
plt.axis('off')

plt.tight_layout()
plt.show()
```



